



EL FIN DEL LOTE: DOMINANDO EL STREAMING DE DATOS

Más allá de la Arquitectura Lambda y los datos aproximados. Un marco conceptual riguroso para arquitecturas de procesamiento continuo.

La Asimetría del Tiempo: El Origen del Caos

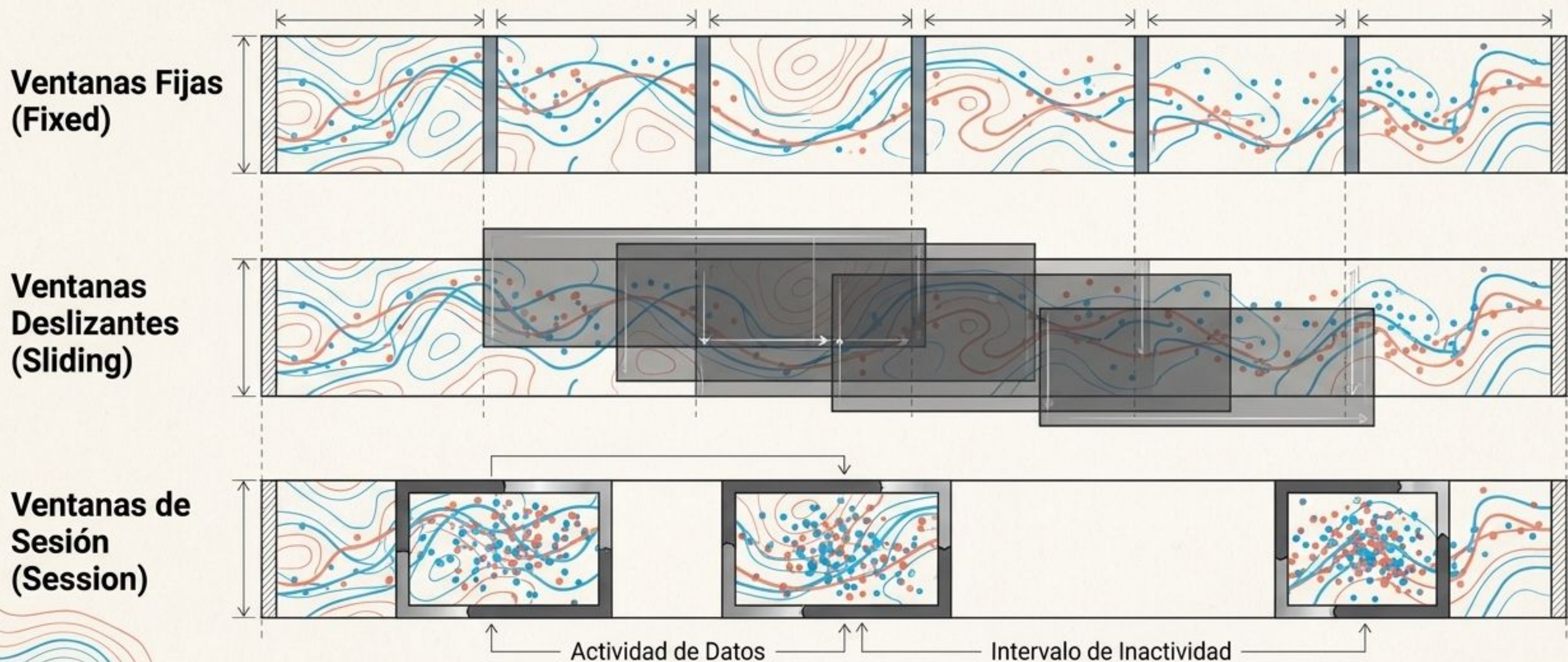
En los sistemas distribuidos del mundo real existe una discrepancia constante entre el momento en que ocurre un evento y el momento en que el sistema lo observa.



- **Tiempo del Evento (Event Time):**
Cuándo ocurrió realmente el evento.
- **Tiempo de Procesamiento (Processing Time):**
Cuándo es observado por el sistema.
- **El Retraso (Time Skew):**
La latencia impredecible introducida por redes, dispositivos móviles offline y limitaciones del sistema.

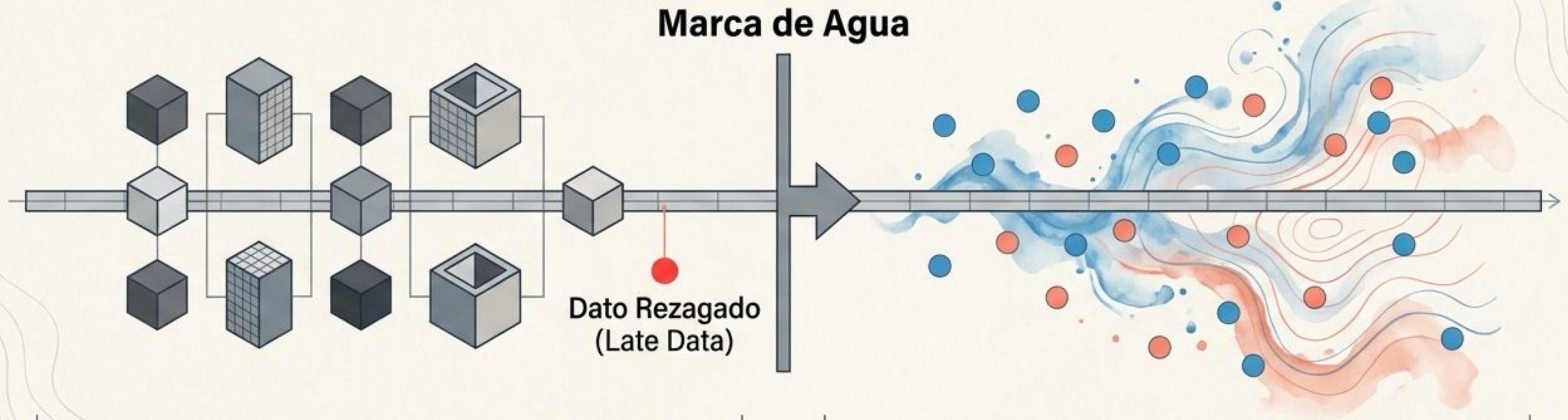
¿DÓNDE en el tiempo del evento? (Ventanas)

Las ventanas agrupan datos no delimitados en bloques finitos basados en su Tiempo de Evento, independientemente de en qué orden lleguen. Las ventanas de sesión son exclusivas del verdadero procesamiento de streaming, ya que son impulsadas dinámicamente por los datos (data-driven).



¿CUÁNDO se materializan? (Marcas de Agua)

Una Marca de Agua es una declaración heurística de completitud: "El sistema asume que todos los datos con tiempos de evento anteriores a X ya han sido observados".



El Problema de ser Muy Lento

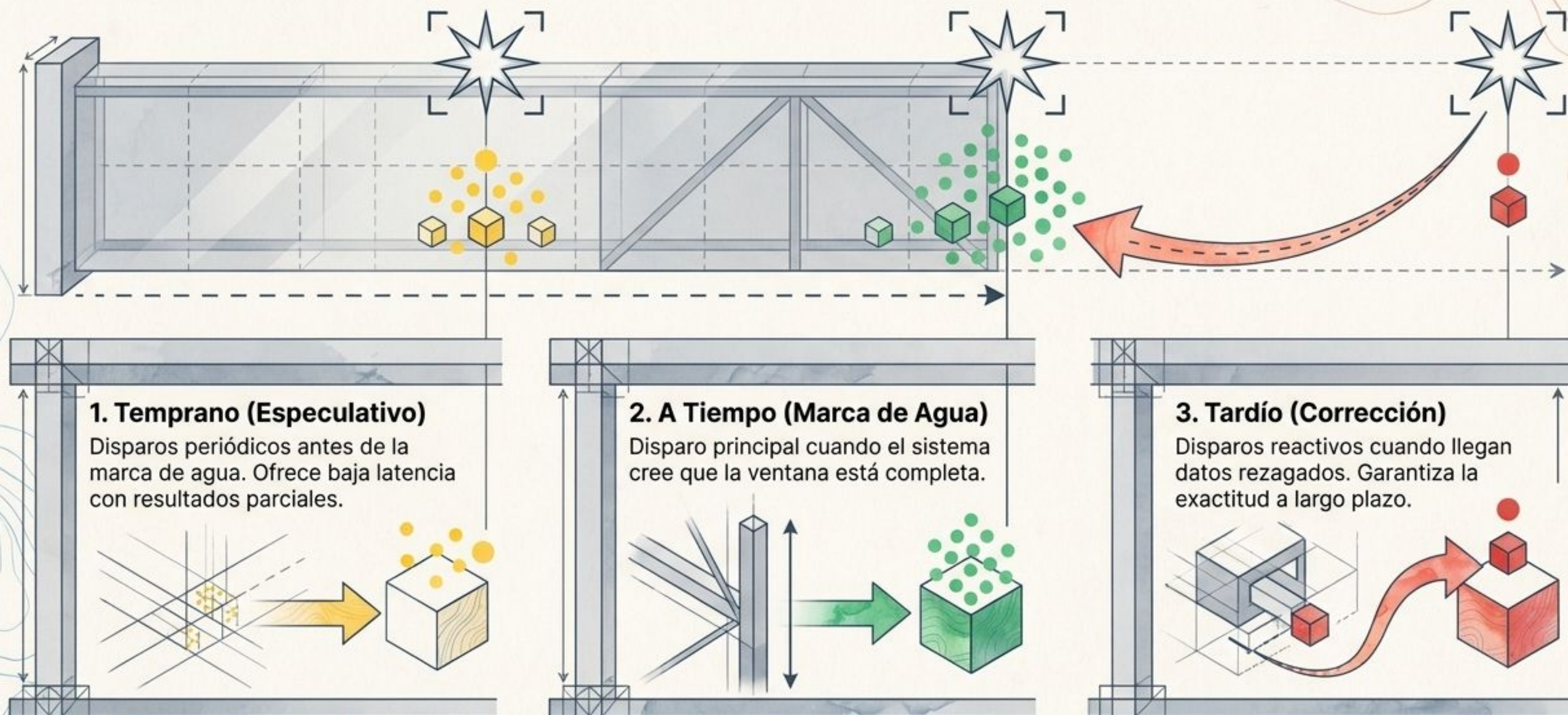
Depender solo de marcas de agua retrasa los resultados (alta latencia) si hay cuellos de botella en la red.

El Problema de ser Muy Rápido

Las heurísticas pueden equivocarse, generando datos rezagados (late data) que arruinan la exactitud.

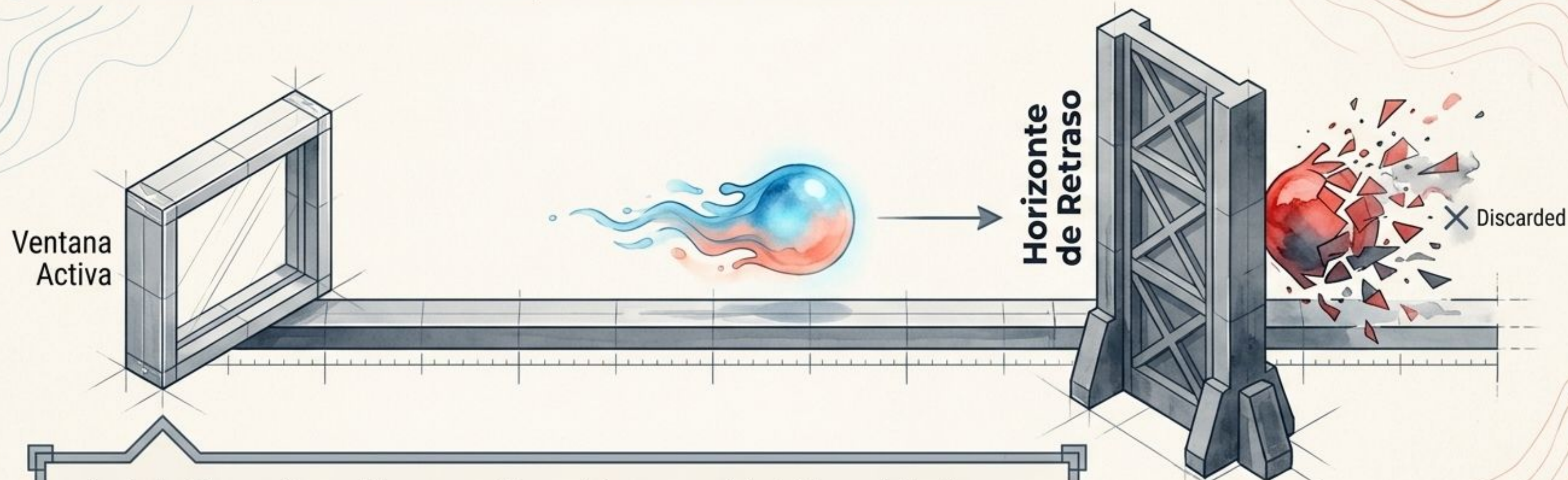
¿CUÁNDO se materializan? (Disparadores)

Los disparadores (Triggers) desvinculan la materialización de resultados de las marcas de agua, permitiendo flujos de trabajo especulativos y correcciones retroactivas.



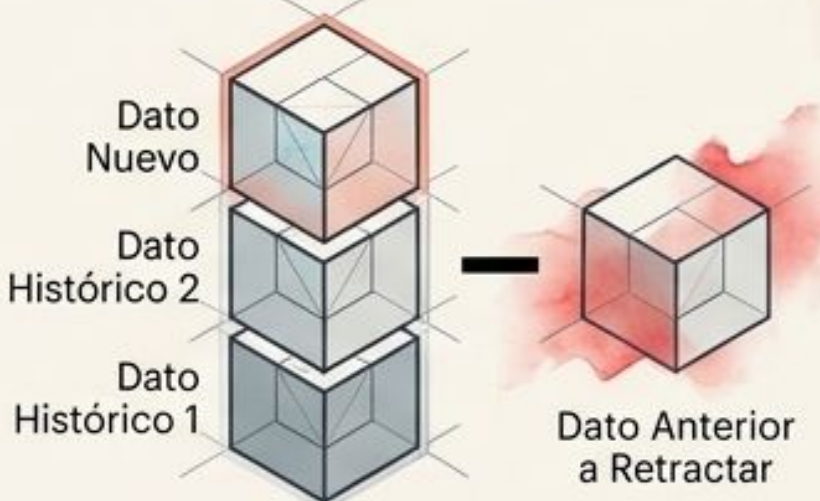
Gestión de Estado: El Horizonte de Retraso

En un sistema de streaming infinito, el almacenamiento de estado físico es finito.
Debemos definir explícitamente cuánto tiempo el sistema debe recordar una ventana.



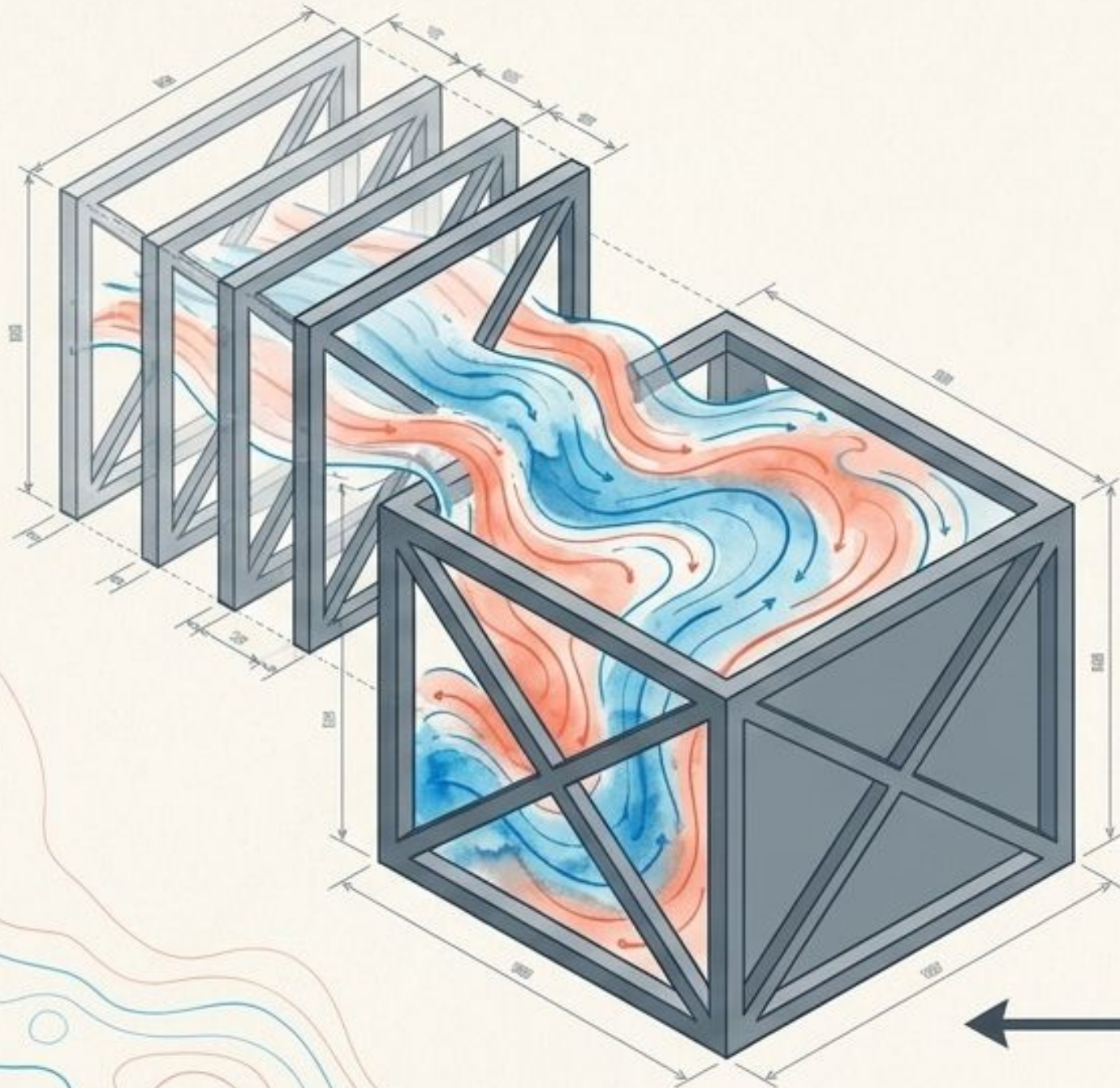
Insight Clave: Al establecer un Allowed Lateness (ej. 24 horas), le damos permiso al motor para limpiar la memoria (Garbage Collection). Cualquier dato que llegue después de este horizonte simplemente se descarta.

¿CÓMO se relacionan los refinamientos? (Acumulación)

Descartar (Discarding)	Acumular (Accumulating)	Acumular y Retractor (Retracting)
 Dato Nuevo 1 Dato Nuevo 2 Dato Nuevo 3	 Dato Nuevo Dato Histórico 2 Dato Histórico 1	 Dato Nuevo Dato Histórico 2 Dato Histórico 1 Dato Anterior a Retractor
Mecánica: Cada disparo emite solo los datos nuevos.	Mecánica: Cada disparo emite el historial completo más lo nuevo.	Mecánica: Emite el valor nuevo Y una orden de eliminación del valor anterior.
Uso: Ideal si el sistema destino ya realiza sus propias sumas/agregaciones.	Uso: Ideal para sobrescribir valores en bases de datos clave-valor (ej. HBase/Cassandra).	Uso: Esencial para reagrupamientos dinámicos y ventanas de sesión.

Síntesis: El Batch es un Caso Especial

La prueba matemática de que el procesamiento continuo es el modelo supremo. Un proceso Batch tradicional es simplemente un pipeline de Streaming configurado de la siguiente manera:



The Batch Equation:

Qué: Transformaciones estándar.

Dónde: Una única Ventana Global que abarca todo el tiempo.

Cuándo: Un único Disparador al final, cuando la fuente de datos finita se agota (Marca de Agua = Infinito).

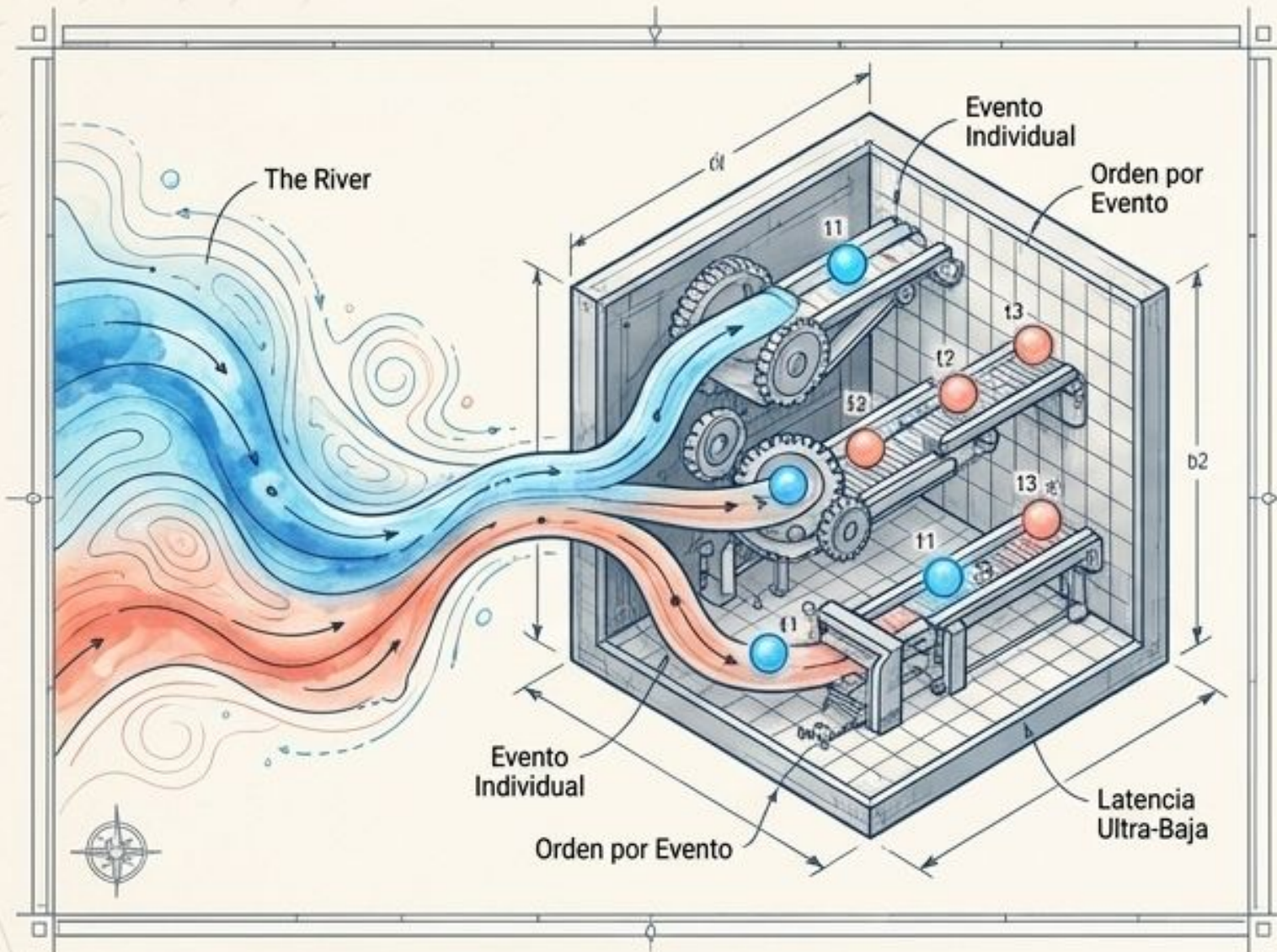
Cómo: Descartar (se emite un solo resultado final).

El Ecosistema: Herramientas para el Modelo Unificado

The Engine Rosetta Stone Matrix

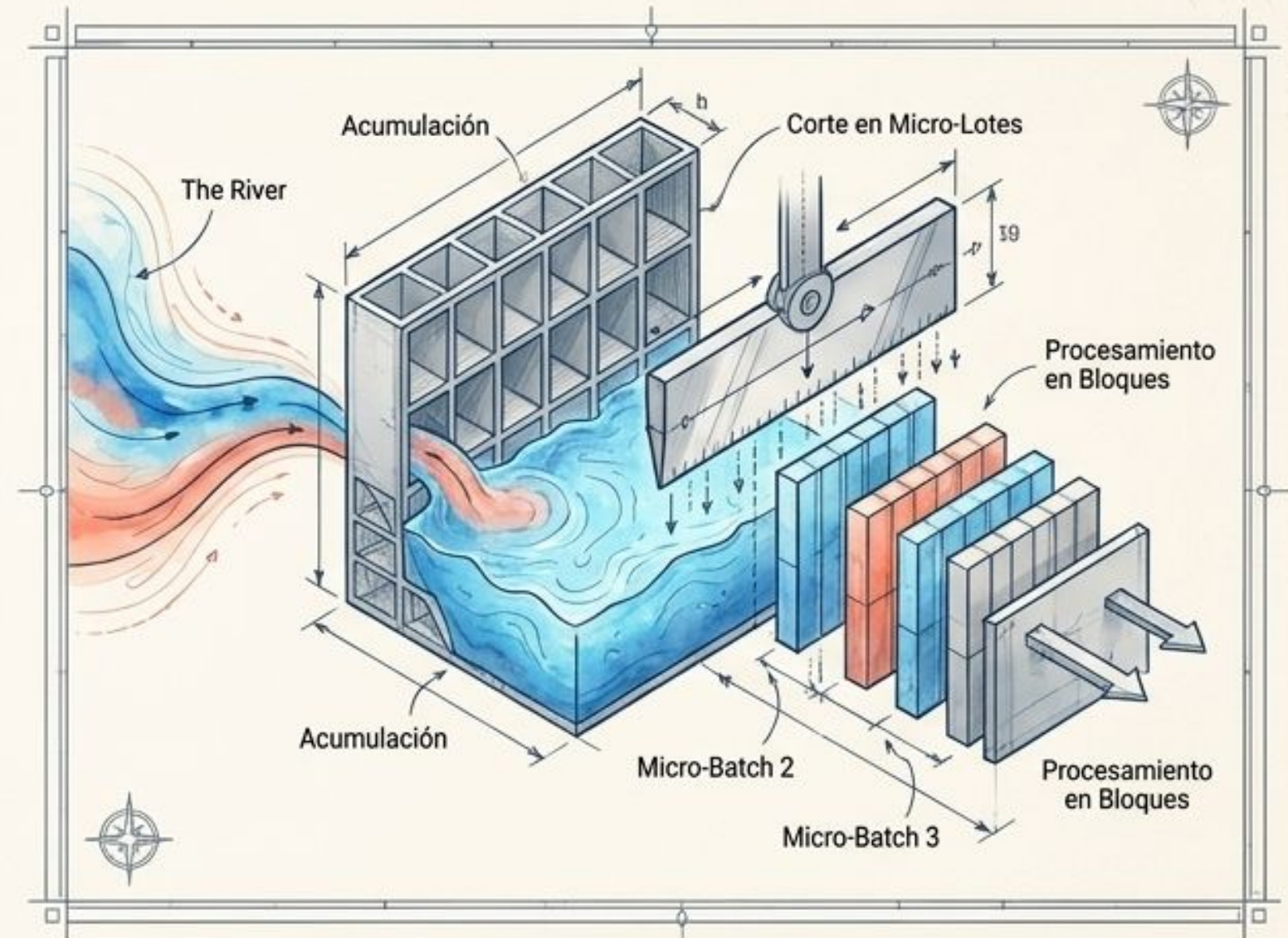
	Filosofía	Ejecución Batch / Streaming	Puntos Fuertes
Apache Flink	Filosofía: Streaming nativo, estado distribuido.	Ejecución Batch: Simulado como un caso especial de streaming.	Puntos Fuertes: Baja latencia real, ventanas complejas, control de estado exacto.
Apache Spark	Filosofía: Micro-batching, análisis unificado.	Ejecución Streaming: Simulada mediante ráfagas rápidas de batch (Spark Streaming / Structured Streaming).	Puntos Fuertes: Ecosistema gigante (MLlib, Spark SQL), adopción masiva.
Apache Beam	Filosofía: Capa de abstracción, portabilidad total.	Ejecución: Agnóstica. Delega la ejecución a un Runner (Flink, Spark, Dataflow).	Puntos Fuertes: Modelo unificado para batch y streaming, independencia del motor.

Flink vs. Spark: La Diferencia Arquitectónica



Flink (Event Time Nativo)

Diseñado desde el núcleo para procesar **cada evento individualmente** y mantener el estado consistente. El estándar de oro para procesamiento de **latencia ultra-baja** y **arquitecturas reactivas complejas**.

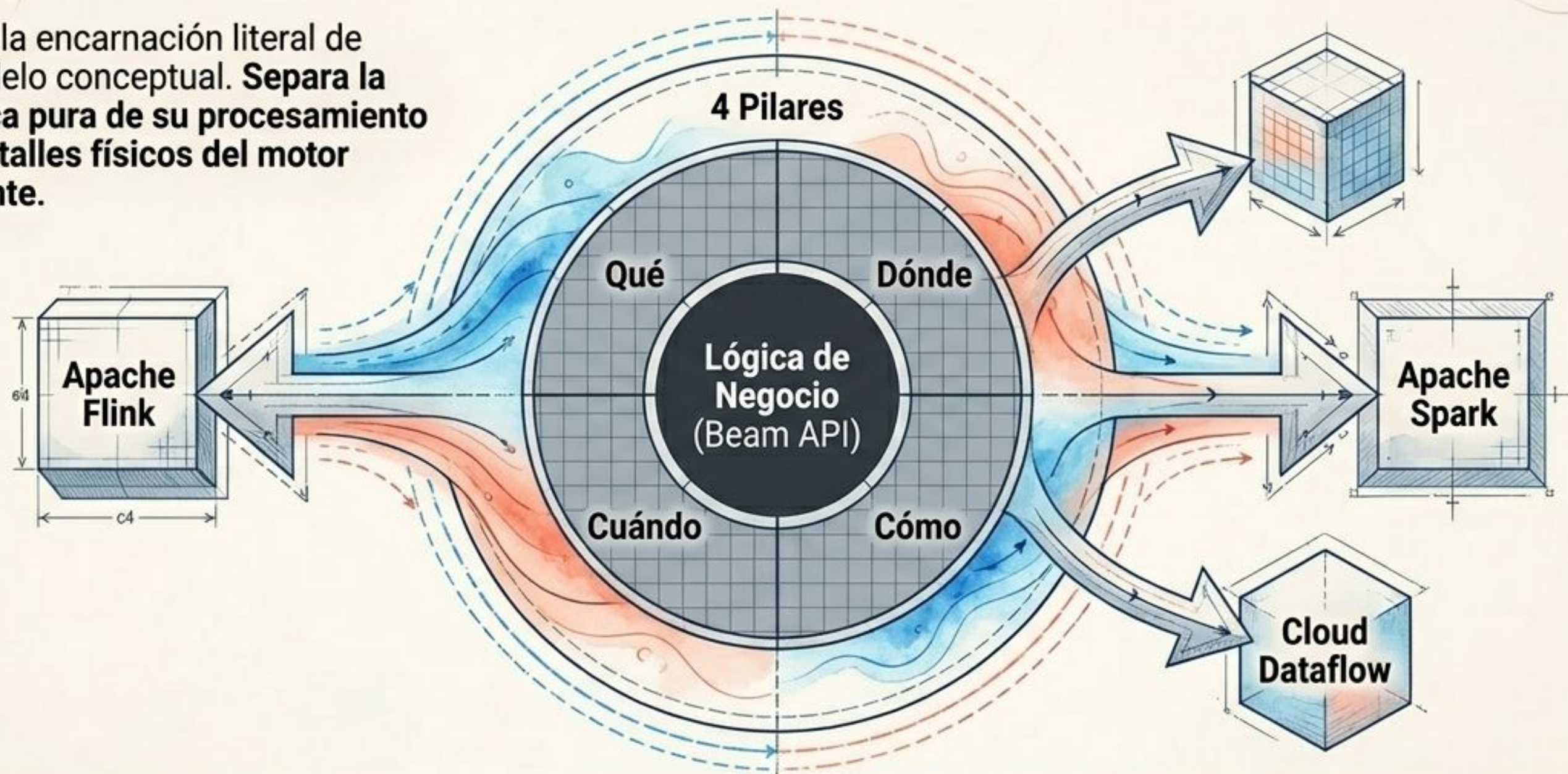


Spark (Ingress Time / Micro-Batch)

Agrupar datos basándose en cuándo son observados por el motor. Excelente para throughput **masivo** y **analítica SQL estructurada**, pero requiere **heurísticas complejas** para imitar el tiempo de evento real.

Apache Beam: Escribir Una Vez, Ejecutar en Cualquier Lugar

Beam es la encarnación literal de este modelo conceptual. **Separa la semántica pura de su procesamiento de los detalles físicos del motor subyacente.**



Conclusión Estratégica: No construya tuberías monolíticas basadas en las limitaciones de un motor específico. Diseñe su arquitectura enfocada en la semántica del tiempo del evento, asegurando exactitud hoy y portabilidad para el futuro.